Lab 4

Mohammadreza (Morez) Tayaranian
Modified by Matthew Mora

ECSE 444, Fall 2025

October 23

Topics

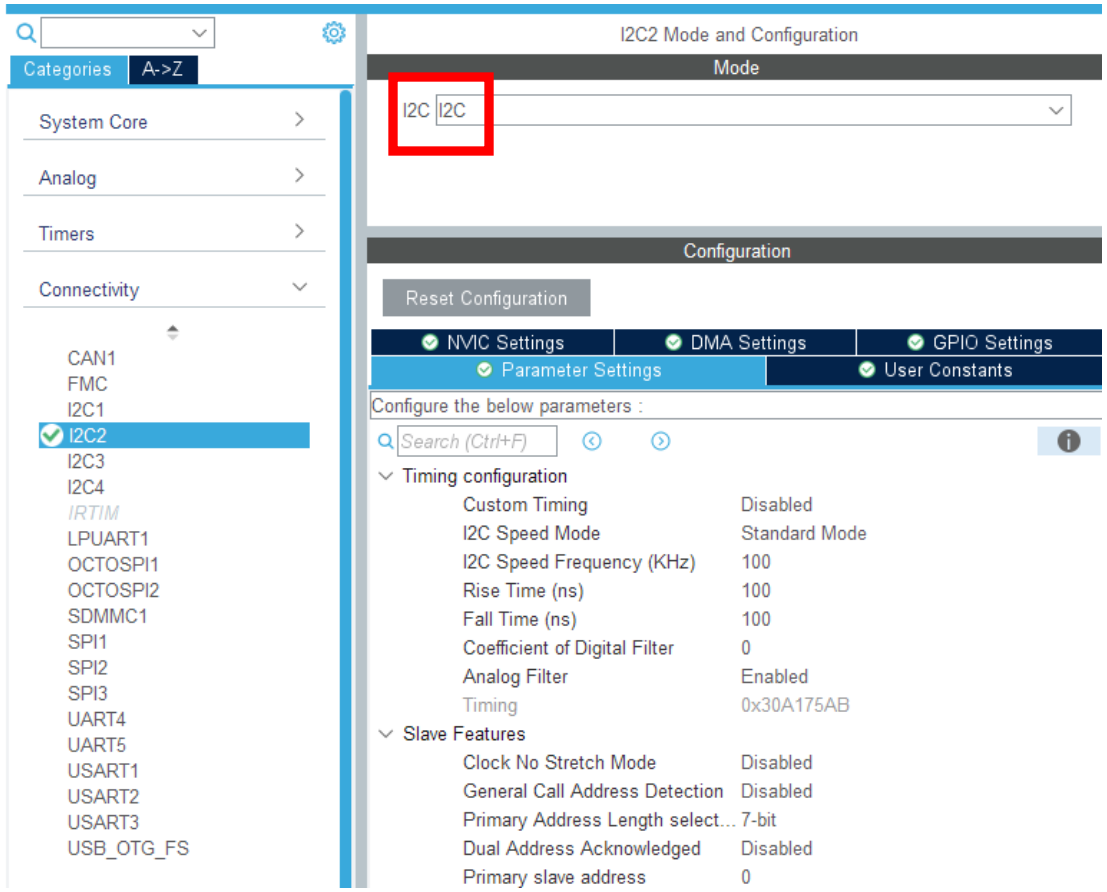
- I^2C
- UART
- OS
- QSPI flash

When you enable OS and nothing works anymore

I^2C

- What sensors do we have?

1. HTS221
   - Temperature and Humidity
2. LIS3MDL
   - Magnetometer
3. LSM6DSL
   - Accelerometer and Gyroscope
4. LPS22HB
   - Pressure



I^2C

- We have BSP drivers that will read the data and convert it for us

1. Copy .h and .c files from

`C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V1.18.1\Drivers\BSP\B-L4S5I-IOT01`

to the project's `Core\Inc` and `Core\Src` folders

I^2C

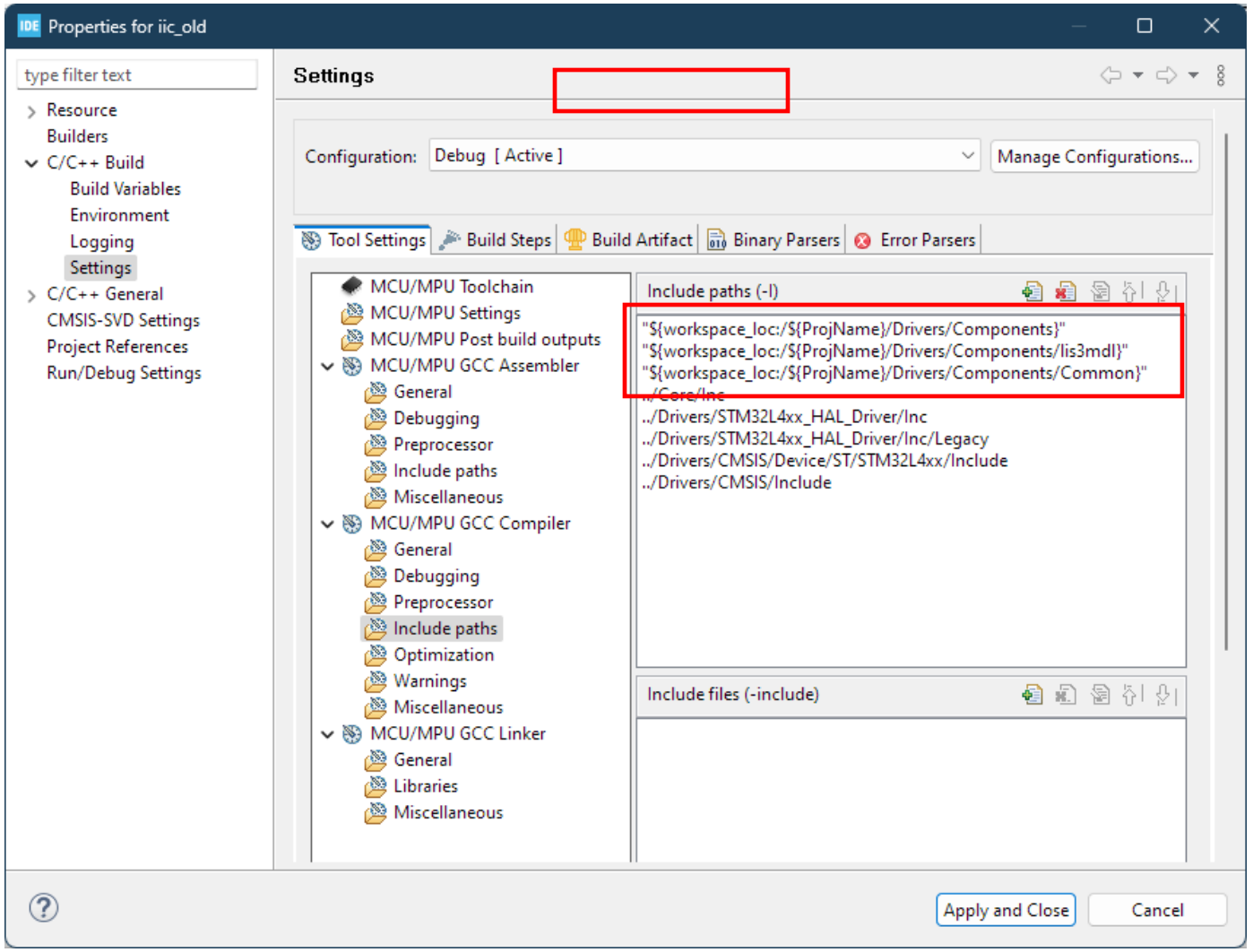
2. Copy the folder

`C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V1.18.1\Drivers\BSP\Components`

to the project's `Drivers` folders

I²C

3. This step must be repeated for every new sensor you want to use. In this example we use LIS3MDL



I^2C

4. Look at the header function you added in step 1 and use the functions to talk with the I2C sensor.

- This is an example for the humidity sensor
  - `BSP_HSENSOR_Init`
  - `BSP_HSENSOR_ReadXXX`

I^2C

- While you probably won't need to know the addresses of each sensor, it can be helpful if you encounter issues.
- Can be found in this document: Discovery kit for IoT node, multi-channel communication with STM32L4+ Series - User manual
- An example from the document (you won't be using this sensor in the lab)

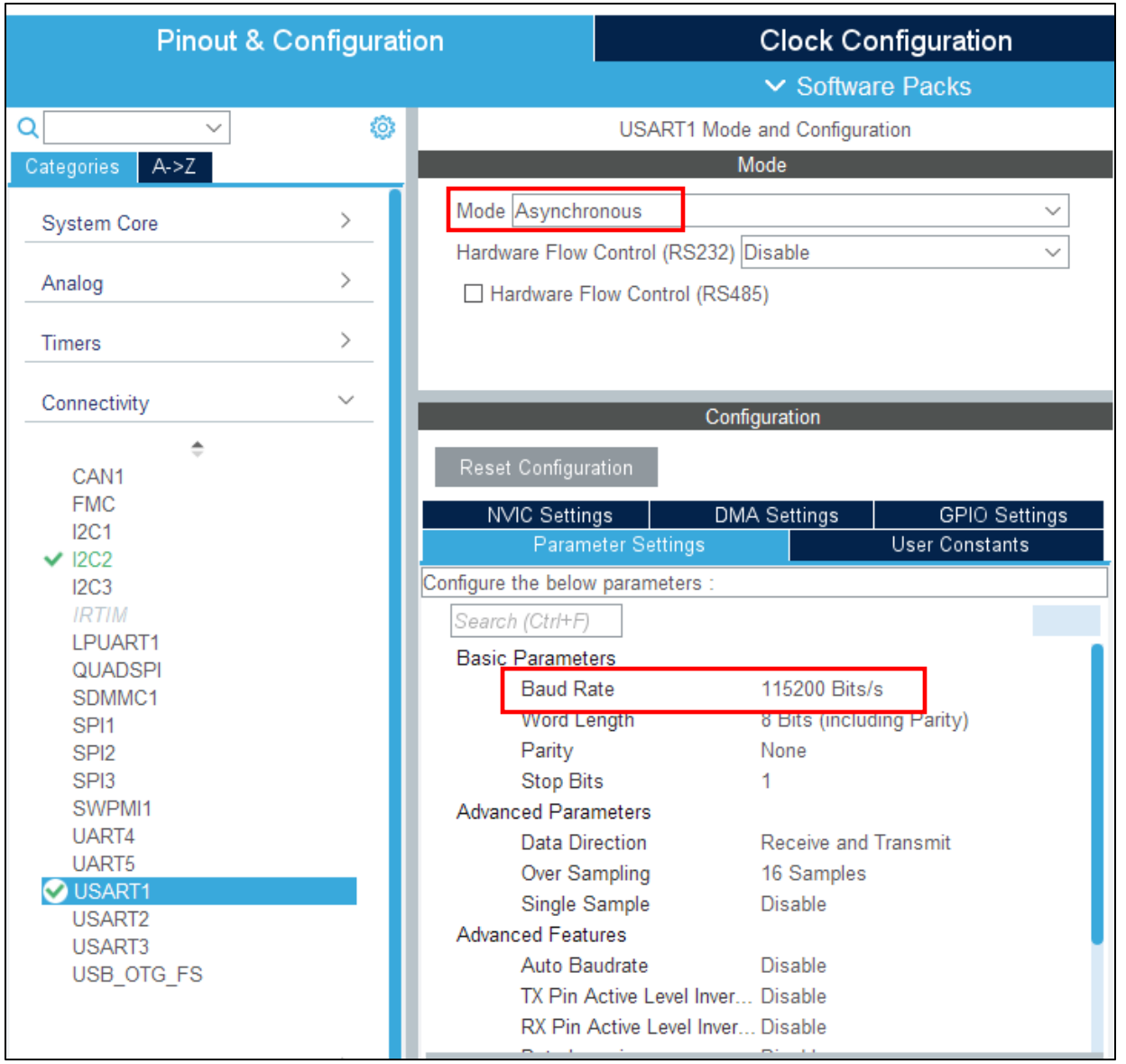
Modules	Description	SAD[6:0] + R/W	I2C write address	I2C read address
VL53L0X	Time-of-Flight ranging and gesture detection sensor	0101001x	0x52	0x53

UART

1. Enable it in ioc

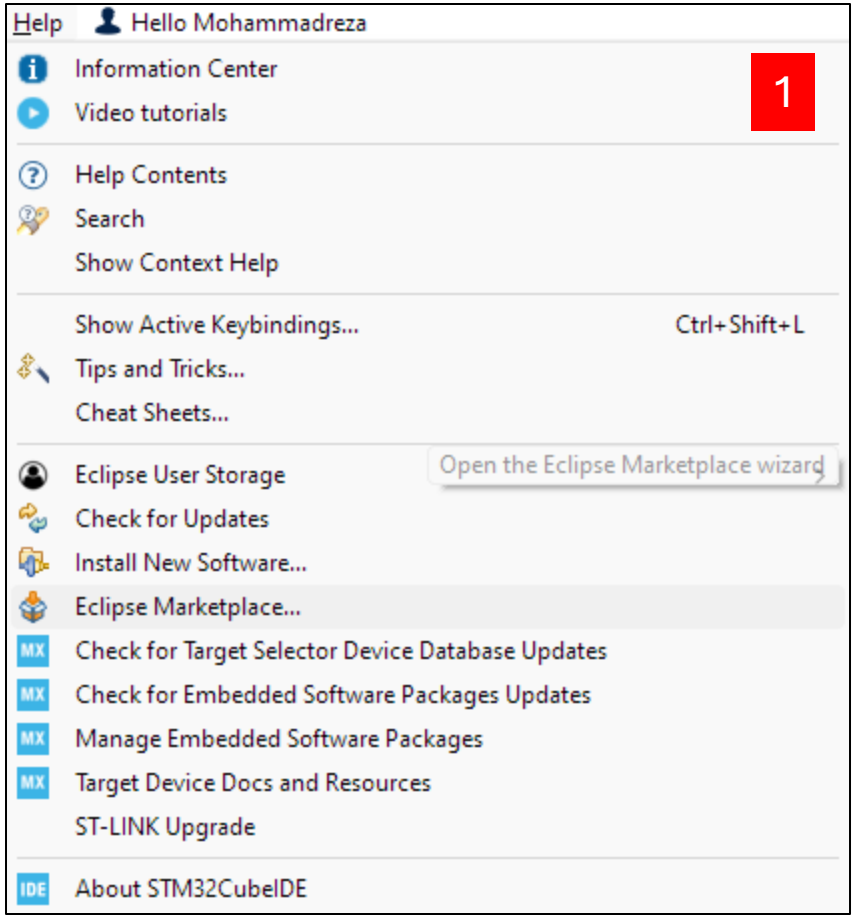
*** Don't forget to make a note of the baud rate, this is important ***

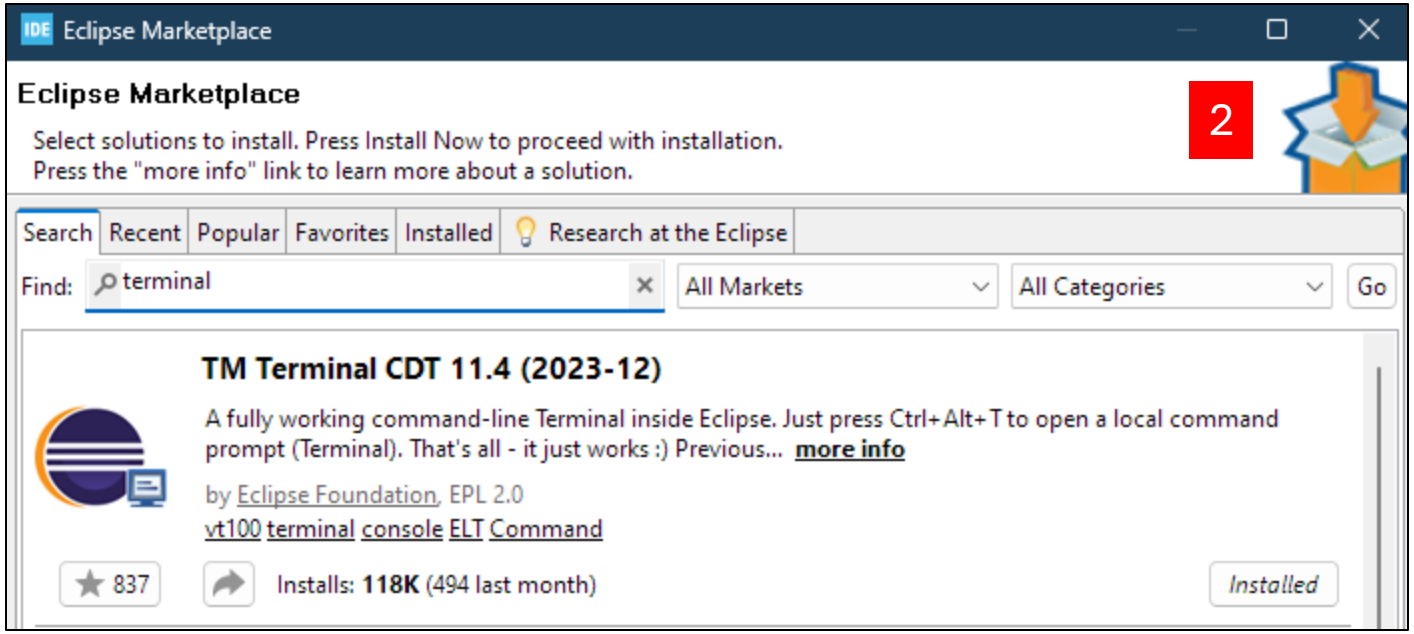
- Basic Parameters
  - Baud Rate 115200 Bits/s
  - Word Length 8 Bits (including Parity)
  - Parity None
  - Stop Bits 1



UART

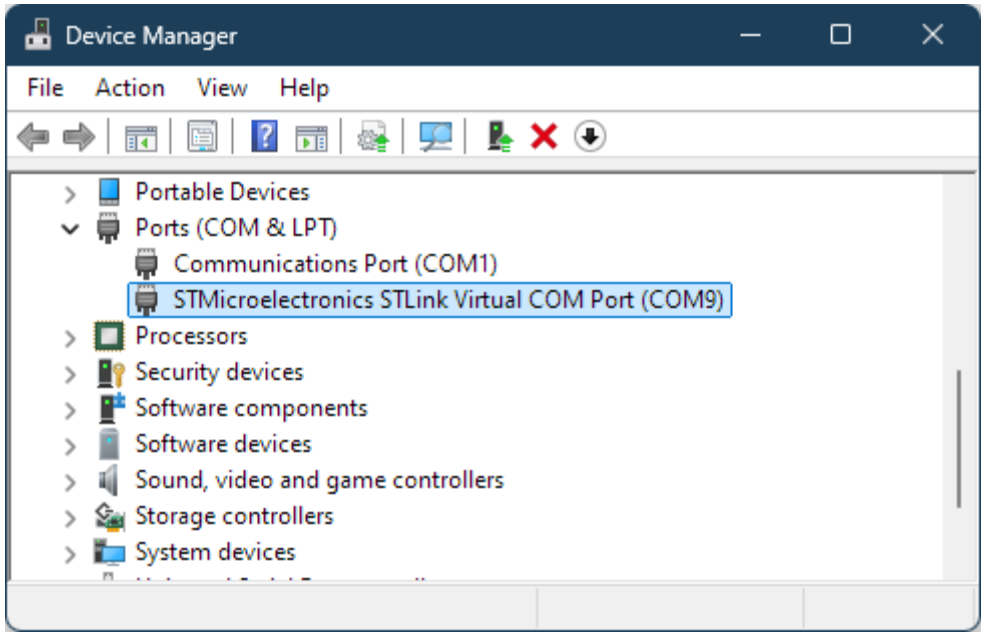
2. Install a terminal

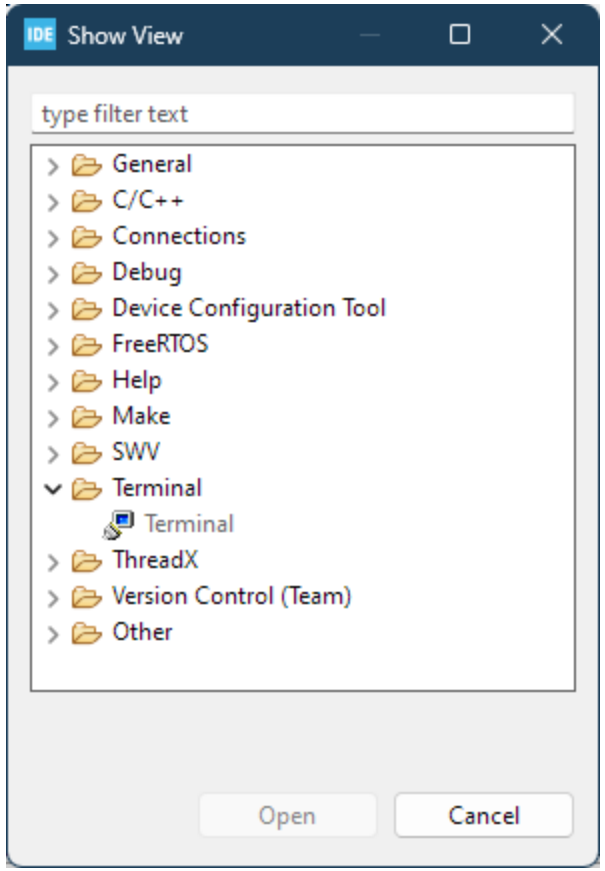




UART

3. Open the terminal
   - Make sure to connect to the correct COM port
   - Also make sure the baud rates match





UART

4. Send data over UART

HAL_UART_Transmit

Function name	**HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)**
Function description	Send an amount of data in blocking mode.
Parameters	• **huart:** UART handle. • **pData:** Pointer to data buffer. • **Size:** Amount of data to be sent. • **Timeout:** Timeout duration.
Return values	• **HAL:** status
Notes	• When FIFO mode is enabled, writing a data in the TDR register adds one data to the TXFIFO. Write operations to the TDR register are performed when TXFNF flag is set. From hardware perspective, TXFNF flag and TXE are mapped on the same bit-field.

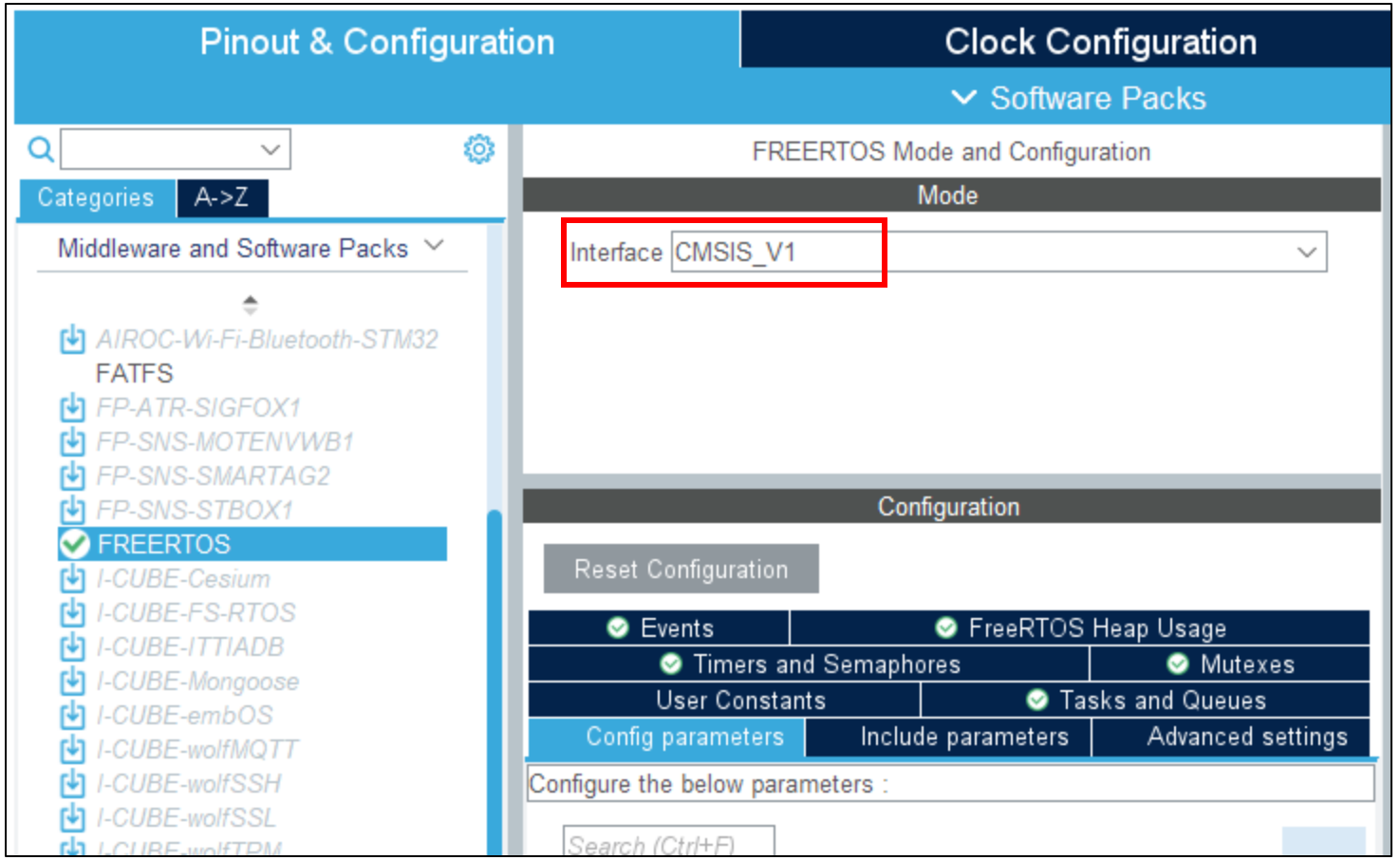
UART

- Example code

```
sprintf(output, "magDataXYZ = %d\r\n", data);
uint16_t len = strlen(output);
HAL_UART_Transmit(&huart1, (uint8_t *)output, len, 100);
```

OS

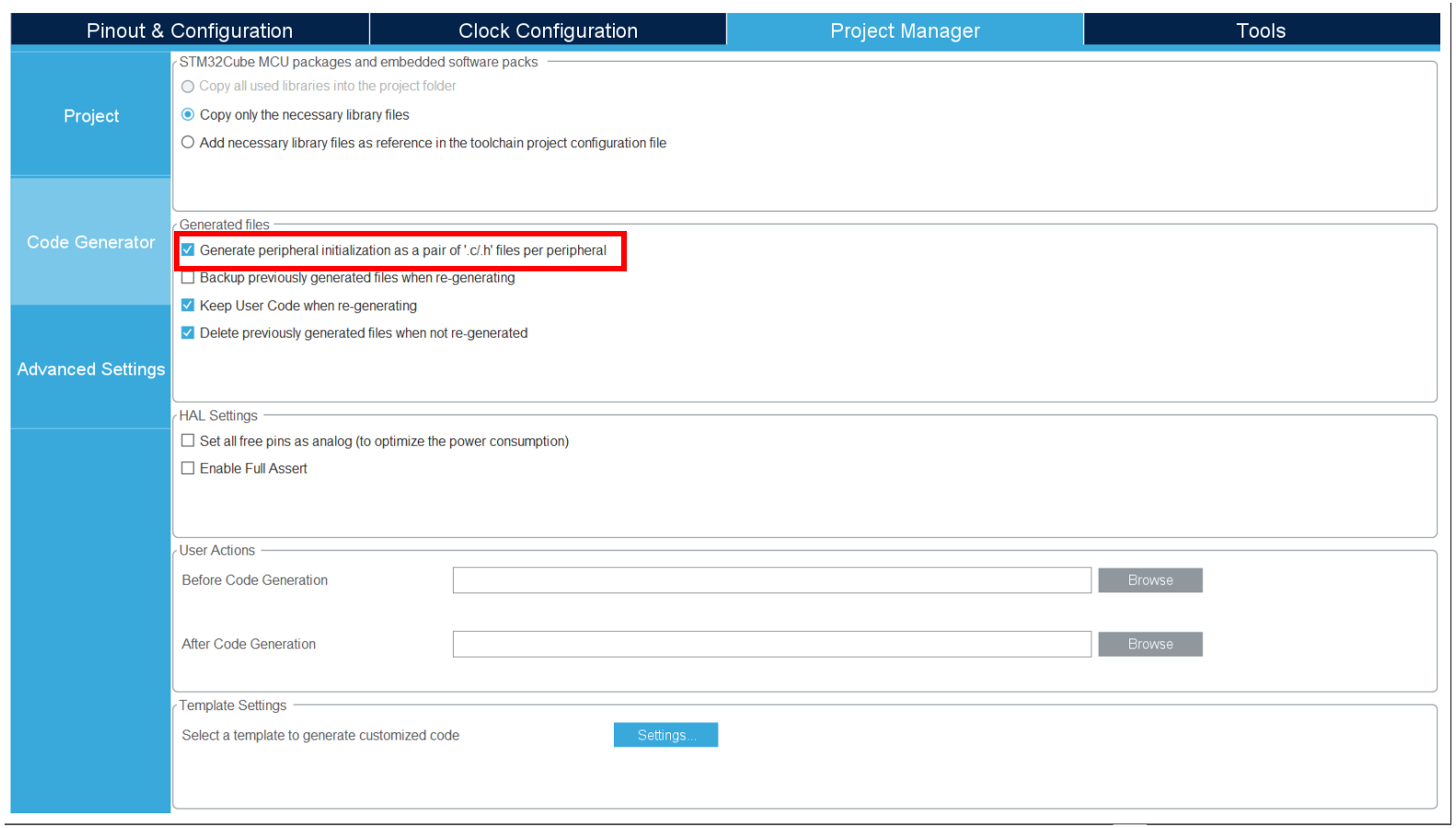
- Enable it in ioc



OS

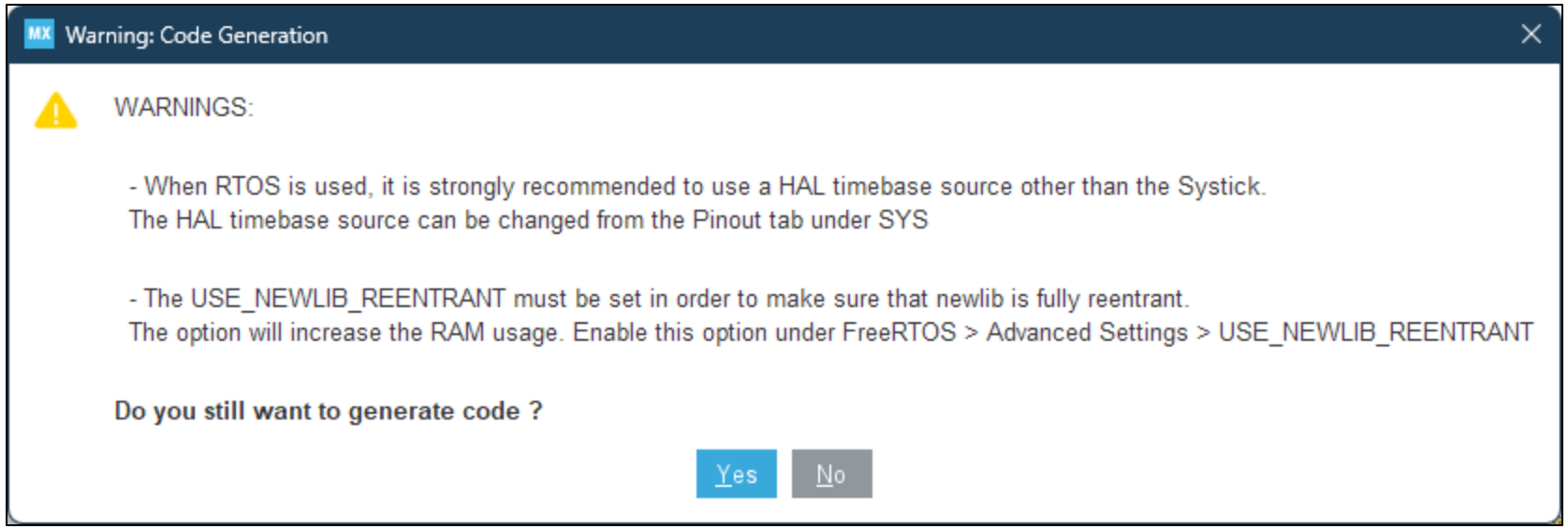
Optional:

checking this box will create separate .c and .h files for each peripheral – it can make finding code cleaner if following good programming practices



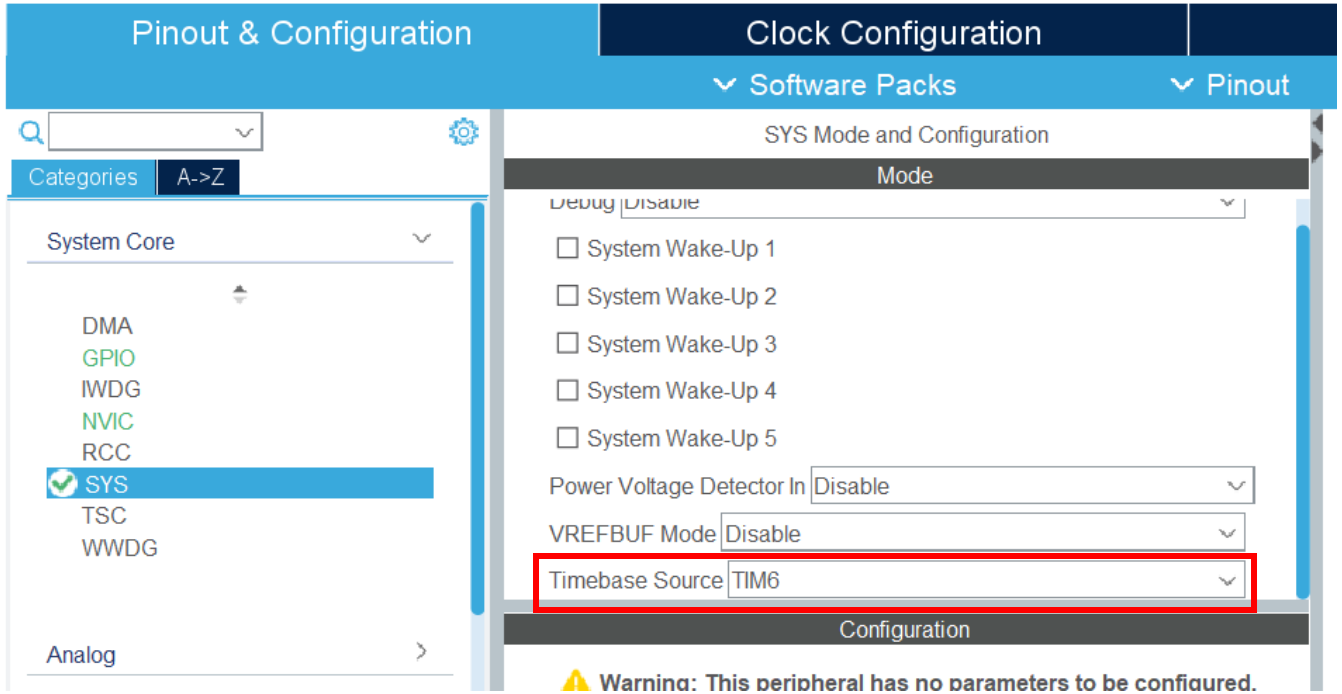
OS

- Uh oh we get a warning!



OS

- Changing the timebase
- osDelay vs HAL_Delay



OS

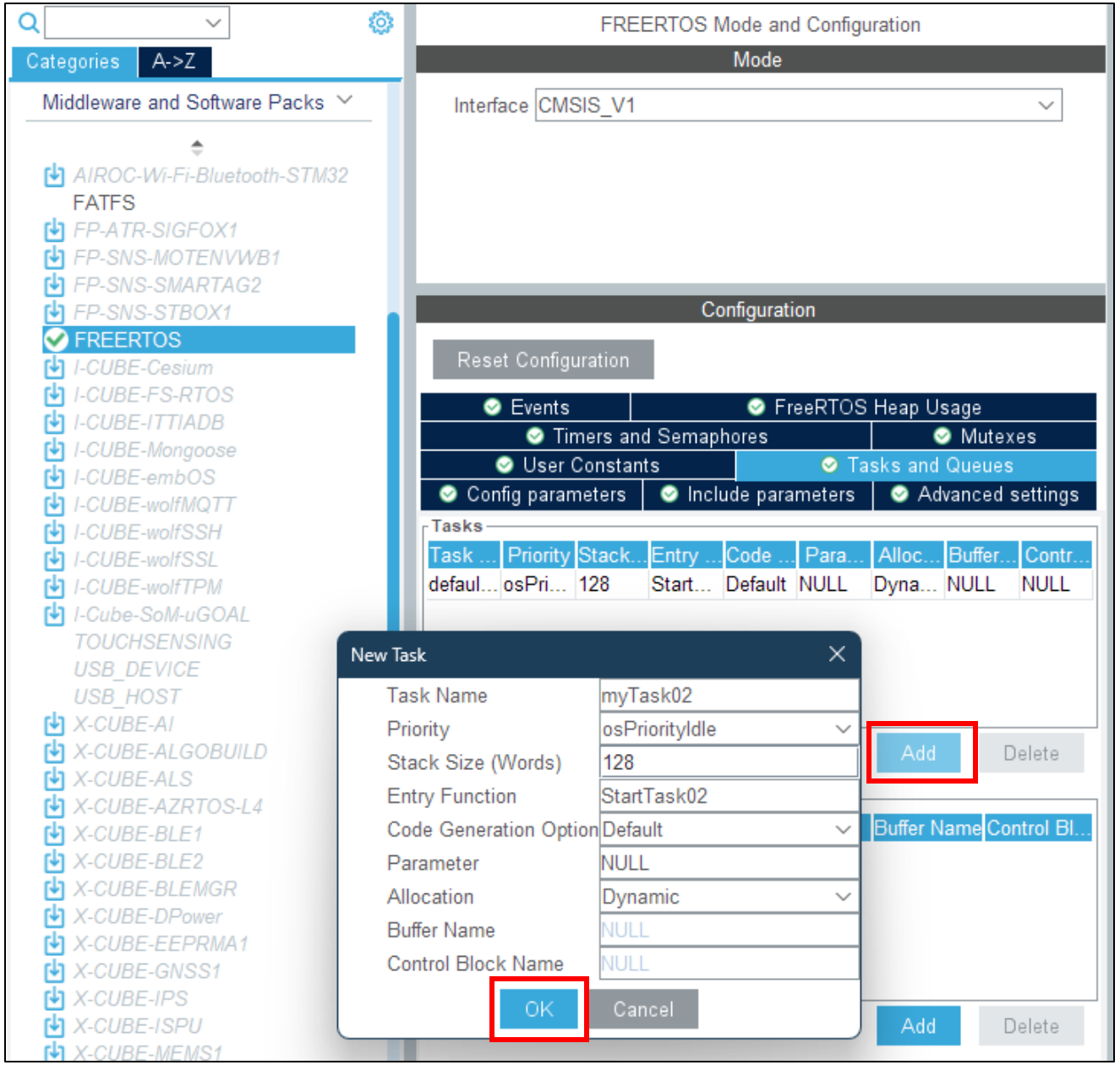
- Main functionality is no longer in the infinite while

```
131   /* Start scheduler */
132   osKernelStart();
133
134   /* We should never get here as control is now taken by the scheduler */
135
136   /* Infinite loop */
137   /* USER CODE BEGIN WHILE */
138   while (1)
139   {
140     /* USER CODE END WHILE */
141
142     /* USER CODE BEGIN 3 */
143   }
144   /* USER CODE END 3 */
```

OS

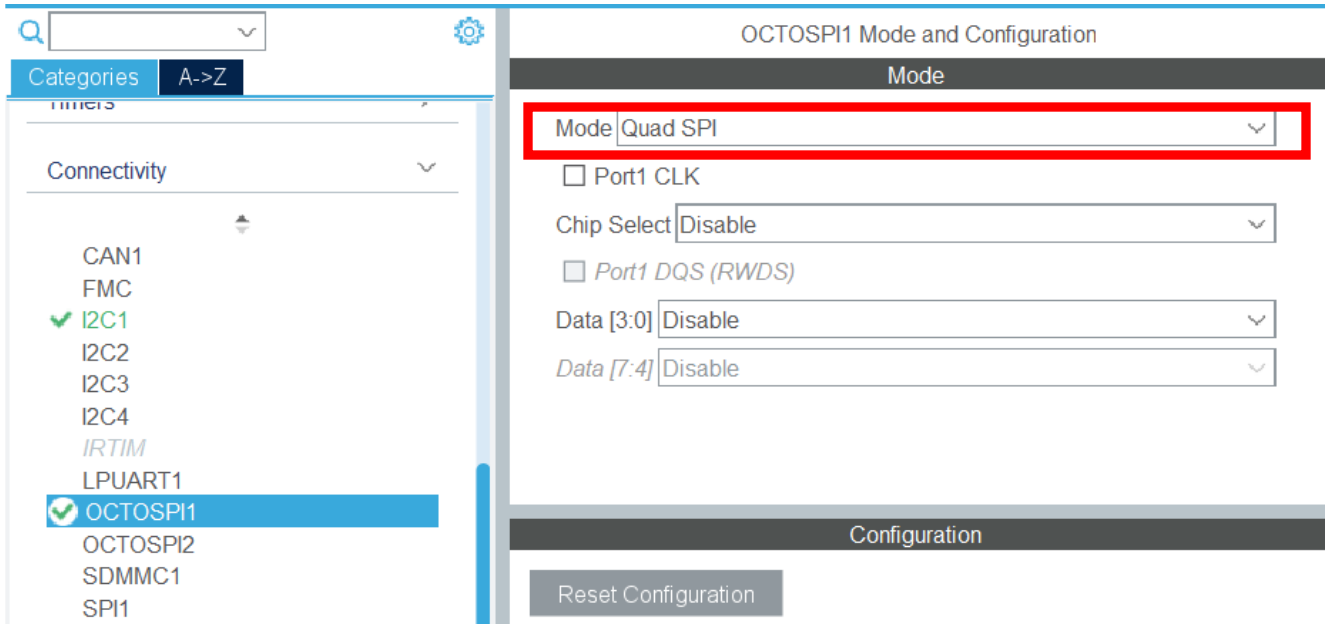
- Put your code in tasks

```
308 /* USER CODE END Header_StartDefaultTask */
309 void StartDefaultTask(void const * argument)
310 {
311   /* USER CODE BEGIN 5 */
312   /* Infinite loop */
313   for(;;)
314   {
315     osDelay(1);
316   }
317   /* USER CODE END 5 */
318 }
```



QSPI

- Set the OCTOSPI1 to Quad SPI mode
- This will not set all (or maybe even any) pins



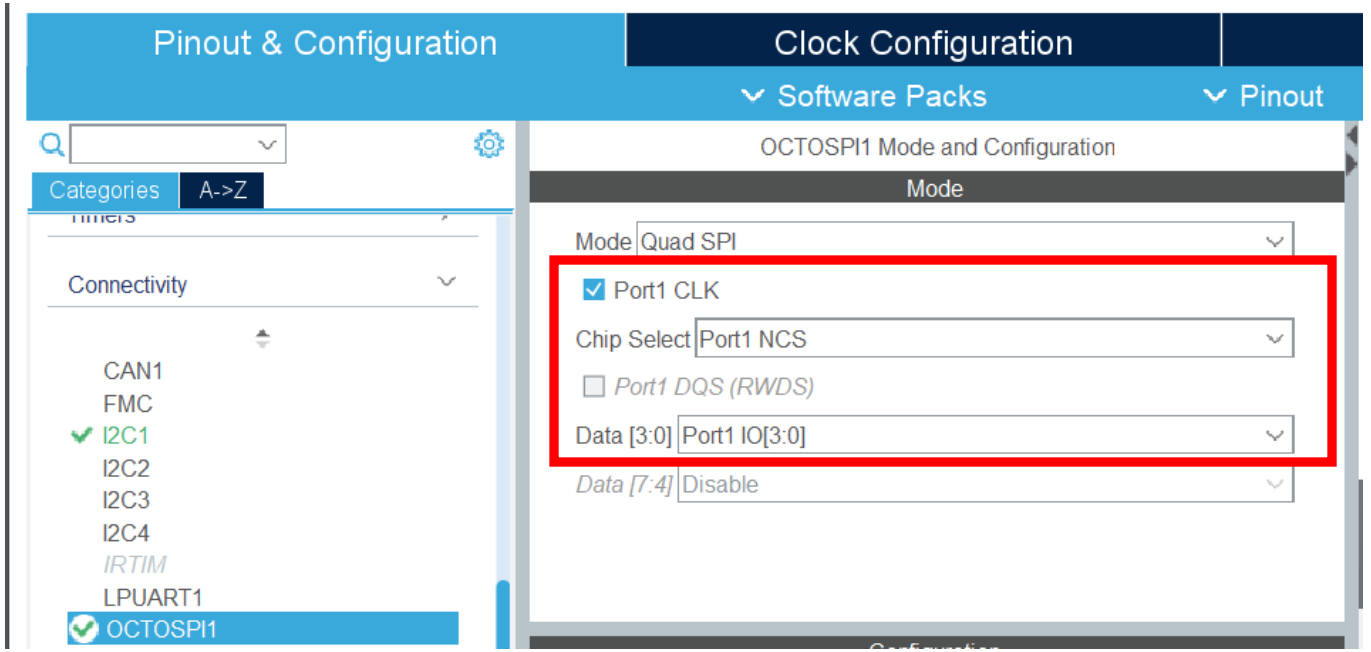
QSPI

- Pinout can be found in this document (for those with the B-L4S5I-IOT01A board): Discovery kit for IoT node, multi-channel communication with STM32L4+ Series - User manual

Pin number	Pin name	Feature / comment	Signal / label
38	PE7	MEMS microphone	DFSDM1_DATIN2
39	PE8	GPIO_Output	ISM43362-RST
40	PE9	MEMS microphone	DFSDM1_CKOUT
41	PE10	QSPI NOR Flash memory	QUADSPI_CLK

QSPI

- Alternatively this selection does the same thing
- Always double check with the datasheet though!



QSPI

- Once again we have BSP files for handling the QSPI operations

1. Copy .h and .c files from

`C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V1.18.1\Drivers\BSP\B-L4S5I-IOT01`

to the project's `Core\Inc` and `Core\Src` folders

QSPI

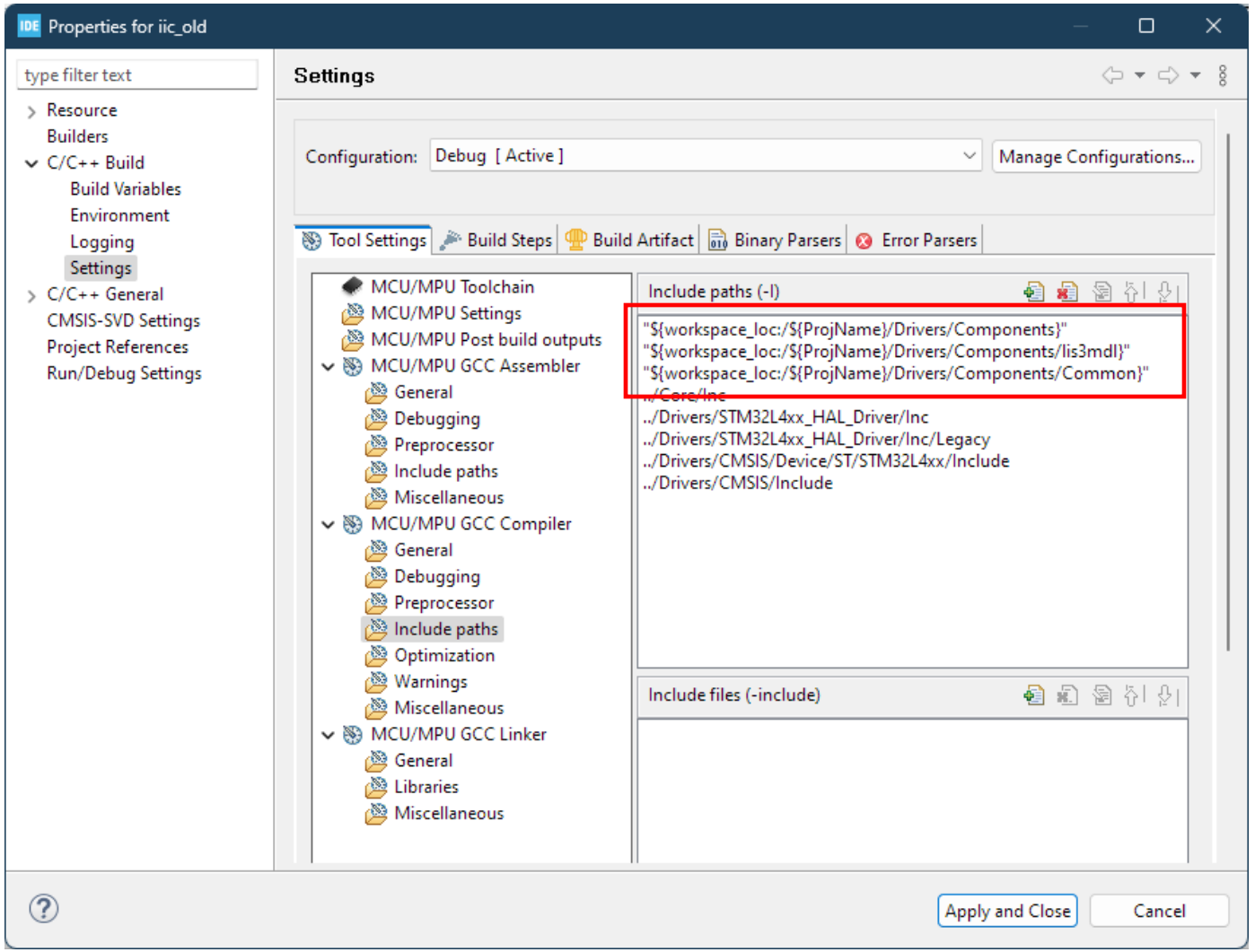
2. Copy mx25r6435f.h from

`C:\Users\John\STM32Cube\Repository\STM32Cube_FW_L4_V1.18.1\Drivers\BSP\Components`

to the project's `Drivers` folders

QSPI

3. Add the files to the linker tree as we did with the sensors in part 1



QSPI

4. Some useful functions you may want to look into

- BSP_QSPI_Init() *don't forget to use this one*
- BSP_QSPI_Write()
- BSP_QSPI_Read()
- BSP_QSPI_Erase_Block()
- BSP_QSPI_Erase_Sector()

*** BSP_QSPI_Erase_Sector() is non-blocking! This means your code will not wait until the function is done executing before continuing ***